

Pokémon Memory Matching App

This document describes the design and implementation of a Pokémon-themed memory matching game built with Flutter and Dart. The application uses Flutter widgets to compose the interface, Dart state variables to manage game behavior, and timer-based logic to control the 1-minute gameplay limit.

1. Project Overview

The app is a mobile memory matching game. The player selects two Pokémon cards at a time. If the Pokémon match, both cards remain visible. If they do not match, the cards flip back after a short delay.

- Theme: Pokémon memory matching game
- Board size: 4 columns × 5 rows, giving 20 total cards
- Matching target: 10 Pokémon pairs
- Time limit: 1 minute per game
- Main interaction: tap a Poké Ball card to reveal a hidden Pokémon
- Winning condition: all pairs are matched before the timer reaches zero
- Losing condition: the 1-minute timer reaches zero before all pairs are matched

2. Application Structure

The project follows a simple Flutter structure. The main implementation is placed inside `lib/main.dart`, while image assets are stored under `assets/images/`.

```
pokemon_memory_match/  
├── lib/  
│   └── main.dart  
├── assets/  
│   └── images/  
│       ├── pokeball.png  
│       ├── pikachu.png  
│       ├── charmander.png  
│       ├── squirtle.png  
│       ├── bulbasaur.png  
│       ├── eevee.png  
│       ├── snorlax.png  
│       ├── jigglypuff.png  
│       ├── meowth.png  
│       ├── psyduck.png  
│       └── gengar.png  
└── pubspec.yaml
```

3. Main Dart Classes

3.1 PokemonMemoryApp

`PokemonMemoryApp` is the root widget of the application. It extends `StatelessWidget` because it only sets up the `MaterialApp`, theme, title, and first page. It does not directly store changing game data.

```
class PokemonMemoryApp extends StatelessWidget {  
  const PokemonMemoryApp({super.key});  
}
```

```

@override
Widget build(BuildContext context) {
  return MaterialApp(
    title: 'Pokemon Memory Match',
    debugShowCheckedModeBanner: false,
    theme: ThemeData(
      useMaterial3: true,
      colorScheme: ColorScheme.fromSeed(seedColor: Colors.red),
    ),
    home: const MemoryGamePage(),
  );
}

```

3.2 PokemonCard Data Model

PokemonCard is a simple data model representing each card on the board. Each card stores its Pokémon name, image path, and current state. The two Boolean fields are important because they control what the UI should display.

```

class PokemonCard {
  final String name;
  final String imagePath;
  bool isFlipped;
  bool isMatched;

  PokemonCard({
    required this.name,
    required this.imagePath,
    this.isFlipped = false,
    this.isMatched = false,
  });
}

```

- name: used to check whether two selected cards match.
- imagePath: used by Image.asset() to display the Pokémon image.
- isFlipped: true when the player has revealed the card.
- isMatched: true after the card has been successfully paired.

3.3 MemoryGamePage and State Management

MemoryGamePage is a StatefulWidget because the game screen changes during play. The timer changes every second, cards flip, pairs are matched, and the number of moves changes. These changing values are stored in the `_MemoryGamePageState` class.

```

class MemoryGamePage extends StatefulWidget {
  const MemoryGamePage({super.key});

  @override
  State<MemoryGamePage> createState() => _MemoryGamePageState();
}

```

4. State Variables

The state variables store the current condition of the game. When these values change inside `setState()`, Flutter rebuilds the related widgets so the screen updates automatically.

Variable	Type	Purpose
----------	------	---------

cards	List<PokemonCard>	Stores all 20 shuffled cards in the current game.
timer	Timer?	Controls the countdown timer.
remainingSeconds	int	Stores how many seconds are left from the 1-minute limit.
moves	int	Counts how many pair attempts the player has made.
firstSelectedIndex	int?	Stores the index of the first selected card.
secondSelectedIndex	int?	Stores the index of the second selected card.
isChecking	bool	Prevents extra taps while the app is checking whether two cards match.
gameOver	bool	Stops input when the game has ended.
gameWon	bool	Records whether the player won the game.

5. Widget Composition and build() Function

In Flutter, the main UI composition function is the `build()` method. It is similar in purpose to a `Compose` function in Jetpack Compose: it describes what the screen should look like based on the current state. Whenever `setState()` is called, Flutter runs `build()` again and updates the UI.

```
@override
Widget build(BuildContext context) {
  return Scaffold(
    body: Container(
      decoration: const BoxDecoration(
        gradient: LinearGradient(
          colors: [Color(0xFF1D2671), Color(0xFFC33764)],
          begin: Alignment.topLeft,
          end: Alignment.bottomRight,
        ),
      ),
    ),
    child: SafeArea(
      child: Column(
        children: [
          buildHeader(warningTime),
          buildInfoPanel(),
          Expanded(child: buildCardGrid()),
          buildFooter(),
        ],
      ),
    ),
  );
}
```

The screen is divided into smaller widget-building helper functions. This improves readability and makes the UI easier to maintain.

- `buildHeader()`: displays the app title and countdown timer.
- `buildInfoPanel()`: displays game information such as moves, matched pairs, and time limit.
- `buildGameCard()`: displays one card, either as a Poké Ball or as a Pokémon image.
- `buildPokeballBack()`: builds the hidden-card view.
- `buildPokemonFace()`: builds the revealed Pokémon-card view.
- `buildFooter()`: displays the restart button.

6. Important Flutter Widgets Used

Widget	Role in the App
MaterialApp	Provides the app-level structure, theme, and home page.
Scaffold	Provides the basic page layout structure.
Container	Used for background gradient, card background, and panel decoration.
SafeArea	Prevents UI elements from overlapping the phone status bar or screen edges.
Column and Row	Arrange widgets vertically and horizontally.
Expanded	Allows the game grid to fill the remaining screen area.
GridView.builder	Efficiently creates the 4 × 5 card grid.
GestureDetector	Detects card tapping.
AnimatedContainer	Animates card decoration changes smoothly.
AnimatedSwitcher	Animates the switch between Poké Ball and Pokémon image.
Image.asset	Loads Poké Ball and Pokémon images from the assets folder.
AlertDialog	Shows win or lose messages.
ElevatedButton.icon	Creates the restart button with an icon.

7. Game Logic Functions

7.1 startNewGame()

This function prepares a new round. It creates two cards for each Pokémon, shuffles them, resets the selected-card indexes, resets the move counter, and starts the timer.

```
void startNewGame() {
  timer?.cancel();

  final List<PokemonCard> generatedCards = [];

  for (final pokemon in pokemonData) {
    generatedCards.add(PokemonCard(
      name: pokemon['name']!,
      imagePath: pokemon['image']!,
    ));
    generatedCards.add(PokemonCard(
      name: pokemon['name']!,
      imagePath: pokemon['image']!,
    ));
  }

  generatedCards.shuffle(Random());

  setState() {
    cards = generatedCards;
    remainingSeconds = totalSeconds;
    moves = 0;
    firstSelectedIndex = null;
    secondSelectedIndex = null;
    isChecking = false;
    gameOver = false;
    gameWon = false;
  });
}
```

```
startTimer();  
}
```

7.2 startTimer()

The timer uses `Timer.periodic()` from Dart. Every second, `remainingSeconds` decreases by 1. When the value reaches 0, the game stops and a losing dialog is displayed.

```
void startTimer() {  
  timer = Timer.periodic(const Duration(seconds: 1), (Timer timer) {  
    if (remainingSeconds <= 1) {  
      timer.cancel();  
      setState(() {  
        remainingSeconds = 0;  
        gameOver = true;  
        gameWon = false;  
      });  
      showEndDialog(  
        title: 'Time is up!',  
        message: 'You ran out of time. Try again!',  
      );  
    } else {  
      setState(() {  
        remainingSeconds--;  
      });  
    }  
  });  
}
```

7.3 onCardTap()

This function handles player interaction. It ignores taps when the game is over, when two cards are currently being checked, or when the tapped card is already flipped or matched. When a second card is selected, the move counter increases by 1.

```
void onCardTap(int index) {  
  if (isChecked || gameOver) return;  
  if (cards[index].isFlipped || cards[index].isMatched) return;  
  
  setState(() {  
    cards[index].isFlipped = true;  
  
    if (firstSelectedIndex == null) {  
      firstSelectedIndex = index;  
    } else {  
      secondSelectedIndex = index;  
      moves++;  
      isChecked = true;  
    }  
  });  
  
  if (firstSelectedIndex != null && secondSelectedIndex != null) {  
    checkMatch();  
  }  
}
```

7.4 checkMatch()

This function compares the names of the two selected Pokémon cards. If the names are the same, the two cards are marked as matched. If they are different, `Future.delayed()` waits briefly, then flips the cards back.

```
void checkMatch() {
  final int firstIndex = firstSelectedIndex!;
  final int secondIndex = secondSelectedIndex!;

  if (cards[firstIndex].name == cards[secondIndex].name) {
    setState(() {
      cards[firstIndex].isMatched = true;
      cards[secondIndex].isMatched = true;
      firstSelectedIndex = null;
      secondSelectedIndex = null;
      isChecking = false;
    });
    checkWin();
  } else {
    Future.delayed(const Duration(milliseconds: 800), () {
      if (!mounted) return;
      setState(() {
        cards[firstIndex].isFlipped = false;
        cards[secondIndex].isFlipped = false;
        firstSelectedIndex = null;
        secondSelectedIndex = null;
        isChecking = false;
      });
    });
  }
}
```

7.5 checkWin()

This function checks whether all cards have been matched. If every card has `isMatched` set to true, the timer stops and the win dialog appears.

```
void checkWin() {
  final bool allMatched = cards.every((card) => card.isMatched);

  if (allMatched) {
    timer?.cancel();
    setState(() {
      gameOver = true;
      gameWon = true;
    });
    showEndDialog(
      title: 'You Win!',
      message: 'Great job! You matched all Pokémon before time ran out.',
    );
  }
}
```

8. UI Design Details

- The background uses a diagonal `LinearGradient` to make the app look more modern and game-like.
- Cards use rounded corners, borders, and shadows to create a polished visual style.

- The card back uses pokeball.png, matching the Pokémon theme.
- When a card is tapped, AnimatedSwitcher changes the displayed child from Poké Ball to Pokémon image.
- Matched cards use a green border to give visual feedback that the pair has been found.
- The timer turns into a warning style when the remaining time is low.
- The info panel displays moves, matched pairs, and time limit to show player progress.

9. Asset Management

All images are loaded locally using `Image.asset()`. The assets folder must be declared in `pubspec.yaml`; otherwise, Flutter cannot find the images at runtime.

```
flutter:  
  uses-material-design: true  
  
assets:  
  - assets/images/
```

The file names in the `assets/images` folder must match the file paths in the Dart code exactly. For example, if the code uses `assets/images/pikachu.png`, the image file must have that exact name and location.

10. How to Run the Project

1. Open the project in IntelliJ IDEA, Android Studio, or VS Code.
2. Make sure Flutter and Dart plugins are installed.
3. Run `flutter pub get` to load dependencies and assets.
4. Select an Android emulator or Chrome as the run target.
5. Run the app using the IDE Run button or the terminal command `flutter run`.

11. Conclusion

This project demonstrates how Flutter and Dart can be used to build a visually attractive and interactive mobile game. Flutter's widget system makes the interface modular and easy to modify, while Dart provides clear support for object-oriented models, timers, state updates, and delayed actions. The final application combines UI design, image assets, event handling, animation, and game logic into one complete memory matching app.